

RAPPORT DE STAGE

FARAMUS François

Dans le cadre du stage de **2^e année** :

Stage effectué du (jj/mm/aa) : 03/06/2024 au 30/08/2024

À : Acquadea,
Lieu-dit Michel Ange, Baleone centre 20167 AFA, France.

Sur le thème : Gestion de la salissure des filets d'aquaculture

Enseignant référent responsable : Catherine Mariojouis

Maître de stage : Chloé Barrier-Loiseau



Engagement de non-plagiat

Principes

- Le plagiat se définit comme l'action d'un individu qui présente comme sien ce qu'il a pris à autrui.
- Le plagiat de tout ou parties de documents existants constitue une violation des droits d'auteur ainsi qu'une fraude caractérisée
- Le plagiat concerne entre autres : des phrases, une partie d'un document, des données, des tableaux, des graphiques, des images et illustrations.
- Le plagiat se situe plus particulièrement à deux niveaux : Ne pas citer la provenance du texte que l'on utilise, ce qui revient à le faire passer pour sien de manière passive. Recopier quasi intégralement un texte ou une partie de texte, sans véritable contribution personnelle, même si la source est citée.

Consignes

- Il est rappelé que la rédaction fait partie du travail de création d'un rapport ou d'un mémoire, en conséquence lorsque l'auteur s'appuie sur un document existant, il ne doit pas recopier les parties l'intéressant mais il doit les synthétiser, les rédiger à sa façon dans son propre texte.
- Vous devez systématiquement et correctement citer les sources des textes, parties de textes, images et autres informations reprises sur d'autres documents, trouvés sur quelque support que ce soit, papier ou numérique en particulier sur internet.
- Vous êtes autorisés à reprendre d'un autre document de très courts passages in extenso, mais à la stricte condition de les faire figurer entièrement entre guillemets et bien sûr d'en citer la source.

Sanction

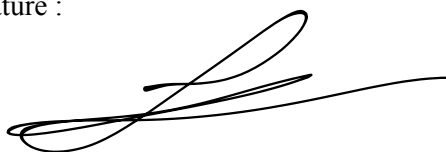
En cas de manquement à ces consignes, la DEVE/le correcteur se réserve le droit d'exiger la réécriture du document sans préjuger d'éventuelles sanctions disciplinaires.

Engagement

Je soussigné François **FARAMUS** reconnais avoir lu et m'engage à respecter les consignes de non-plagiat.

A Ajaccio le 31/08/2024

Signature :



Remerciements

Je tiens tout d'abord à remercier **Mme Chloé Barrier-Loiseau** , ma maître de stage, qui occupe la fonction de responsable production au sein de la société Acquadea, et qui m'a accueilli comme stagiaire dans son service. Il m'a beaucoup appris sur les enjeux de l'aquaculture et sur ses missions au quotidien.

Durant mon stage, j'ai eu la chance de pouvoir bénéficier des conseils de **M. Marc Frichou** et **M. Guillaume Boissier** qui sont respectivement chef de site et ingénieur production et je voudrais les remercier pour le temps qu'ils m'ont accordé et leur aide précieuse.

Je souhaite également adresser mes remerciements les plus sincères aux équipes de la ferme marine des Sanguinaires qui m'ont beaucoup apporté tout au long de mon stage.

Je souhaite bien sûr aussi remercier mon enseignante référente, **Mme Catherine Mariojouis** qui a facilité mes démarches et a assuré un suivi tout au long de mon stage.

Résumé

L'encrassement biologique des filets en aquaculture constitue un défi majeur, influençant directement la santé des poissons et l'efficacité des infrastructures. Ce rapport présente le développement et l'évaluation d'un algorithme de traitement d'image conçu pour quantifier l'encrassement des filets à partir de photographies sous-marines. L'algorithme, qui repose sur des techniques de segmentation et d'analyse de la structure des filets, a été testé sur des images issues de la ferme marine des Sanguinaires, gérée par Acquadea. Les tests réalisés ont montré que l'algorithme était capable de surmonter différents défis liés au traitement d'image, et les premières évaluations suggèrent une cohérence entre les notations manuelles et celles produites par l'algorithme. Cependant, des limitations subsistent, notamment dans la détection des zones fortement encrassées, ce qui suggère des pistes d'amélioration. Ce travail contribue à une meilleure compréhension de l'encrassement des filets et propose des outils pour améliorer les pratiques d'entretien en aquaculture.

Biological fouling of aquaculture nets poses a major challenge, directly impacting fish health and infrastructure efficiency. This report presents the development and evaluation of an image processing algorithm designed to quantify net fouling based on underwater photographs. The algorithm, which utilizes segmentation and net structure analysis techniques, was tested on images from the Sanguinaires marine farm, operated by Acquadea. The tests demonstrated that the algorithm could overcome various challenges related to image processing, and initial evaluations suggest consistency between manual ratings and those generated by the algorithm. However, limitations remain, particularly in detecting heavily fouled areas, suggesting areas for improvement. This work contributes to a better understanding of net fouling and proposes tools to improve maintenance practices in aquaculture.

Table des matières

INTRODUCTION.....	6
1. Présentation de l'entreprise.....	6
2. Contexte géographique.....	6
3. Contexte institutionnel et politique.....	8
I- CAGES FLOTTANTES ET ENTRETIENS DES FILETS.....	10
1.L'encrassement biologique des filets:.....	12
2.Impacts sur la Structure des Cages:.....	12
3.Impacts sur la Santé des Poissons:.....	13
II- L'ÉLABORATION DE L'ALGORITHME.....	17
1. Sélection par l'utilisateur de la partie de l'image qui l'intéresse.....	19
2. Correction de la distorsion.....	19
1. Conversion des Points en Coordonnées Flottantes.....	19
2. Calcul de la Largeur et de la Hauteur de la Région Sélectionnée.....	19
3. Définition des Points de Destination.....	21
4. Création de la Matrice de Transformation.....	21
3. Le "débrumage".....	23
1. Modification du canal de luminosité ('v').....	23
2. Reconstruction de l'image avec les modifications.....	25
4. La segmentation de l'encrassement biologique.....	25
1. Initialisation de l'algorithme.....	25
2. Application de l'algorithme de K-means.....	25
3. Reconstruction de l'image segmentée.....	27
5. L'analyse de la structure du filet.....	27
1. Extraction des canaux de couleur.....	27
2. Seuil pour créer l'image binaire.....	29
3. L'érosion et de la dilatation.....	29
4. Trouver les contours.....	31
5. Création des rectangles englobants:.....	31
6. L'ajustement des rectangles.....	31
6. Estimation du pourcentage du filet recouvert.....	37
1. Boucle sur les rectangles pour calculer le pourcentage de recouvrement.....	37
2. Calcul du pourcentage de recouvrement total de la portion.....	37
III-ANALYSE DES RÉSULTATS DE L'ALGORITHME SUR DIVERS FILETS.....	39
1.Explication de la démarche.....	39
2.Analyse des Résultats : Adaptabilité et Limites de l'Algorithme.....	41
IV- PISTES D'AMÉLIORATION:.....	45
CONCLUSIONS.....	46

INTRODUCTION

1. Présentation de l'entreprise

Le groupe Gloria Maris, fondé par Philippe Riera, est un acteur clé de l'aquaculture en Europe, reconnu pour ses pratiques respectueuses de l'environnement et son engagement en faveur de la qualité et de la durabilité. À travers ses différentes filiales, telles que Aquanord, France Turbot, Palma d'Oro, et Acquadea, le groupe se distingue par une production haut de gamme de poissons, en privilégiant le bien-être animal et le respect des écosystèmes marins.

Parmi les autres filiales du groupe, Aquanord, situé à Gravelines, se distingue par sa capacité à produire 1800 tonnes de poissons par an (Agep, s. d.), tandis que France Turbot est spécialisé dans l'élevage du turbot Label Rouge. Enfin, Palma d'Oro, en Sardaigne, et le site Ichthus de Gravelines, contribuent à l'expansion internationale du groupe en produisant respectivement des maigres et daurades, et des alevins de bars pour l'export.

Acquadea créée en 1992, produit trois espèces principales : la dorade royale (*Sparus aurata*), le loup de mer (*Dicentrarchus labrax*) et le maigre (*Argyrosomus regius*), avec une production totale annuelle de 800 tonnes de poissons (Agep, s. d.). L'entreprise se distingue par des labels de qualité reconnus (label rouge, friend of the sea, aquaculture de nos régions, etc.) assurant la satisfaction de ses clients et le respect des standards environnementaux. Le personnel d'Acquadea se compose d'une équipe diversifiée composée d'une douzaine d'ouvrier aquacole, deux ingénieurs spécialisés, 4 commerciaux, un comptable, une RH et de tous les membres de la maintenance, de la logistique ainsi que du conditionnement et un directeur général qui supervise également les autres filiales du groupe.

Grâce à cette diversité de sites et de spécialisations, Gloria Maris Groupe continue d'être un leader national dans le domaine de l'aquaculture, avec une approche axée sur l'excellence, le savoir-faire et le respect de l'environnement.

2. Contexte géographique

La ferme marine des Sanguinaires d'Acquadea est située dans la baie d'Ajaccio, en Corse, une région renommée pour sa beauté naturelle et sa biodiversité exceptionnelle.



Figure 1. Vue aérienne de la ferme marine des Sanguinaires(Marcelin, 2023)



A



B



C

Figure 2. Photographie des 3 espèces de poissons élevées dans la ferme marine des Sanguinaires. A: le loup de mer (*Dicentrarchus labrax*), B: la dorade royale (*Sparus aurata*), C: le maigre (*Argyrosomus regius*)

Classée NATURA 2000, le site est régulièrement visité par la Station de Recherche Océanographique et Sous-marines (STARESO) pour assurer un suivi environnemental strict, garantissant ainsi la préservation du milieu marin. Cette zone offre des conditions idéales pour l'aquaculture, avec une eau de mer propre et une température stable, propices à l'élevage des poissons. Les îles Sanguinaires à l'ouest protègent la ferme des vents dominants, tandis que la proximité avec les marchés locaux et les infrastructures de transport facilite la distribution des produits. Cependant, la distance entre le site et le siège à Baleone, à environ 45 minutes en voiture, pose des défis logistiques pour le personnel, notamment pour les ingénieurs alternant entre le bureau et la ferme mais surtout pour le transport du poisson pêché sur le site qui doit transiter jusqu'au conditionnement.

3. Contexte institutionnel et politique

Acquadea opère dans un cadre institutionnel rigoureux, respectant les réglementations nationales et européennes en matière de sécurité alimentaire, de protection de l'environnement et de bien-être animal. L'entreprise est régulièrement auditée par des organismes de contrôle tels que l'Agence Nationale de Sécurité Sanitaire de l'Alimentation, de l'Environnement et du Travail (ANSES) et l'Autorité Européenne de Sécurité des Aliments (EFSA)(Agep, s. d.). Les politiques européennes et nationales favorisent le développement durable et la préservation des écosystèmes marins, offrant des subventions et programmes de soutien aux entreprises aquacoles engagées dans des pratiques durables, dont bénéficie Acquadea.

Dans ce contexte, où la qualité des infrastructures est essentielle pour maintenir les standards de production, l'état des filets demande une attention particulière. Leur performance et leur durabilité sont cruciales pour offrir un environnement optimal aux poissons et limiter les coûts liés à l'entretien et au remplacement. Cependant, un des plus grands défis auxquels ces filets font face est l'encrassement biologique, un phénomène naturel qui peut avoir de lourdes conséquences pour l'aquaculture. Pour répondre à ces défis, il est essentiel de développer des méthodes précises pour mesurer et comparer cet encrassement, afin d'améliorer les stratégies d'entretien. C'est dans cette optique que se pose la question centrale de ce rapport : **comment mesurer et comparer efficacement l'encrassement biologique des filets en aquaculture pour améliorer les modes de production ?**

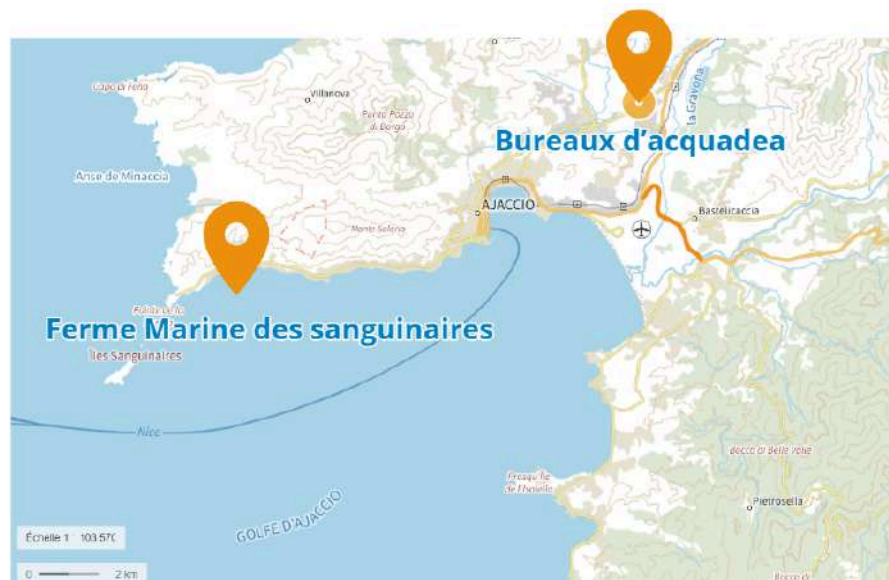


Figure 3. Carte de la localisation des bureaux et de la ferme des Sanguinaires



Figure 4. Photographies de cages de la ferme

I- CAGES FLOTTANTES ET ENTRETIENS DES FILETS

Les cages flottantes sont des structures utilisées en aquaculture pour l'élevage de poissons en milieu ouvert, généralement en mer ou dans de grands plans d'eau. Elles sont constituées d'un cadre flottant maintenu à la surface de l'eau par des flotteurs, auquel est attaché un filet submergé formant une enceinte où les poissons sont élevés.

La ferme marine des Sanguinaires comprend environ 60 cages. Ce nombre varie constamment en fonction des nouveaux arrivages, des mises en vente, et des divisions de certaines cages en plusieurs lots. Ces opérations de tri répartissent les poissons selon leur taille : queues, moyens, et têtes de lot, dans différentes cages.

Trois types de cages existent sur le site, différenciées par leur diamètre : 12 m, 16 m, et 19 m. La taille de la cage est attribuée en fonction de la biomasse totale du lot, les plus grandes cages étant destinées aux biomasses les plus importantes. Un arrivage classique d'alevins commence généralement dans des cages de 12 m et évolue vers des cages de 16 m, puis 19 m au cours de leur cycle de croissance.

Les filets varient également d'une cage à l'autre, tant par la taille des mailles que par le matériau utilisé pour les fils. Sur la ferme, la taille des mailles peut aller de 8 mm pour les plus petits poissons à 25 mm pour les plus gros. Les filets peuvent être fabriqués en nylon ou en polyéthylène, ce dernier offrant une plus grande résistance. Les filets sont composés de mailles carrées sans nœuds aux coins, ce qui évite aux poissons de se blesser en s'y frottant.

Le matériau utilisé pour les filets des maigres et des loups est le polyamide (nylon), réputé pour sa bonne résistance à la traction et son élasticité (Conides et al., 2023). Cependant, les dorades, seul poisson omnivore du site, ont tendance à brouter et à ronger ce qu'elles trouvent à leur portée, notamment le cordage et les filets. Par conséquent, il a été décidé d'utiliser des filets en polyéthylène de la marque Dyneema pour les dorades. Ce matériau est connu pour sa résistance accrue à l'abrasion, aux UV, et aux morsures, offrant une protection mécanique supérieure (Conides et al., 2023).



Figure 5. Schéma (Cardia, 2015) de la structure d'une cage flottante telle qu'on les trouve dans la ferme des Sanguinaires. La ralingue du bas est plombée, on retrouve aussi le système de lestage avec des chaînes maintenue par des cordes le long de la cage et le lest de cône qui est attaché à la partie la plus basse du filet. Ce système permet au filet d'être maintenu à la verticale, en donnant le maximum de volume aux poissons.

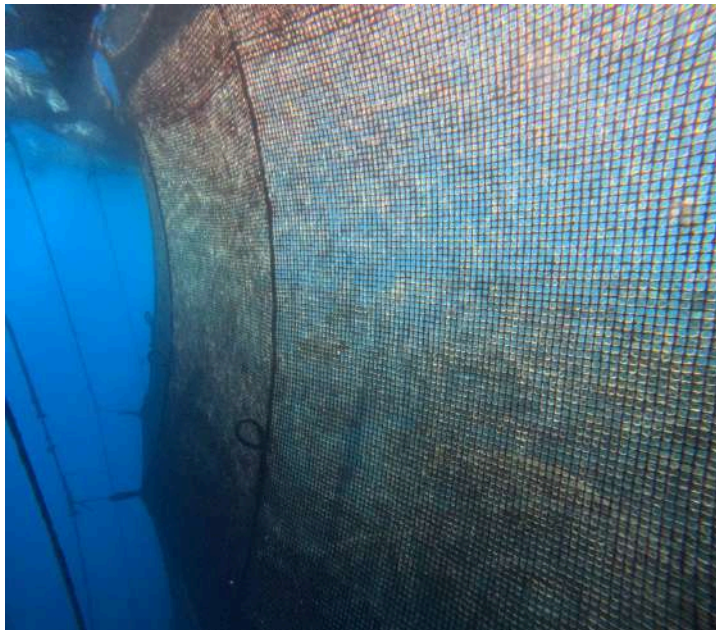


Figure 6. Vue sous marine d'un filet et d'une partie de son système de lestage.

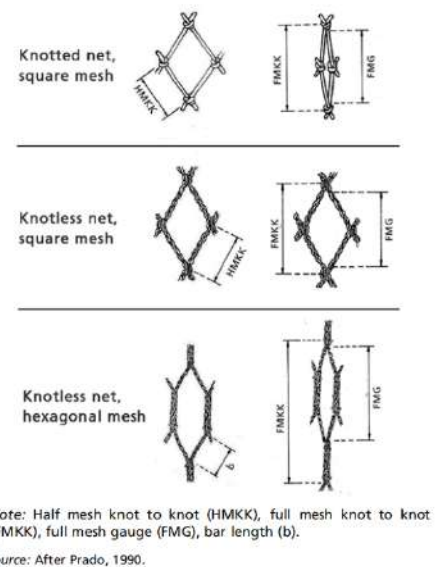


Figure 7. Schéma (Cardia, 2015) des différentes mailles existantes pour les filets d'aquacultures. Ceux de la ferme sont de la forme de la deuxième.

1.L'encrassement biologique des filets:

Comme nous venons de le voir, une cage flottante est un assortiment complexe de composants immergés avec des filets, des flotteurs, des cordes, des lests, etc. Toutes ces structures constituent des surfaces propices à l'encrassement biologique.

Ce que l'on entend par encrassement biologique ou “biofouling” en anglais représente la multitude d'animaux marins et d'algues qui peuvent s'installer sur ces surfaces immergées. L'encrassement des filets est un problème majeur dans le monde de l'aquaculture en milieu ouvert et peut avoir des répercussions économiques considérables. Les coûts économiques directs du contrôle des salissures biologiques pour l'industrie de l'aquaculture sont considérables, avec des estimations prudentes de 5 à 10 % des coûts de production attribués aux salissures biologiques (Lane et Willemsen 2004). Les méthodes traditionnelles de contrôle incluent le nettoyage fréquent des filets, l'utilisation d'antifoulants chimiques et, plus récemment, le recours à des méthodes de contrôle biologique. Ces estimations ne prennent pas en compte les effets indirects de la salissure des filets sur la production.

2.Impacts sur la Structure des Cages:

L'encrassement des filets entraîne une augmentation significative des forces de traînée sur les filets. En effet, les forces induites par les courants sur un filet encrassé peuvent être jusqu'à 12,5 fois supérieures à celles sur un filet propre. Cette surcharge peut déformer les filets et réduire le volume effectif des cages de 45 à 80%(Dürr & Thomason, 2009), ce qui compromet la qualité de l'eau à l'intérieur des cages en limitant l'échange d'eau. De plus, les filets encrassés augmentent la charge statique sur les filets jusqu'à 200 fois, ce qui peut entraîner des déformations et fissures et donc des échappées de poissons, (Dürr & Thomason, 2009).

3.Impacts sur la Santé des Poissons:

Un filet encrassé réduit l'échange d'eau à travers les cages, ce qui diminue la disponibilité en oxygène dissous et peut augmenter les niveaux d'ammoniac. Ces conditions stressent les poissons et peuvent accroître leur susceptibilité aux maladies (Greene & Grizzle, 2007).

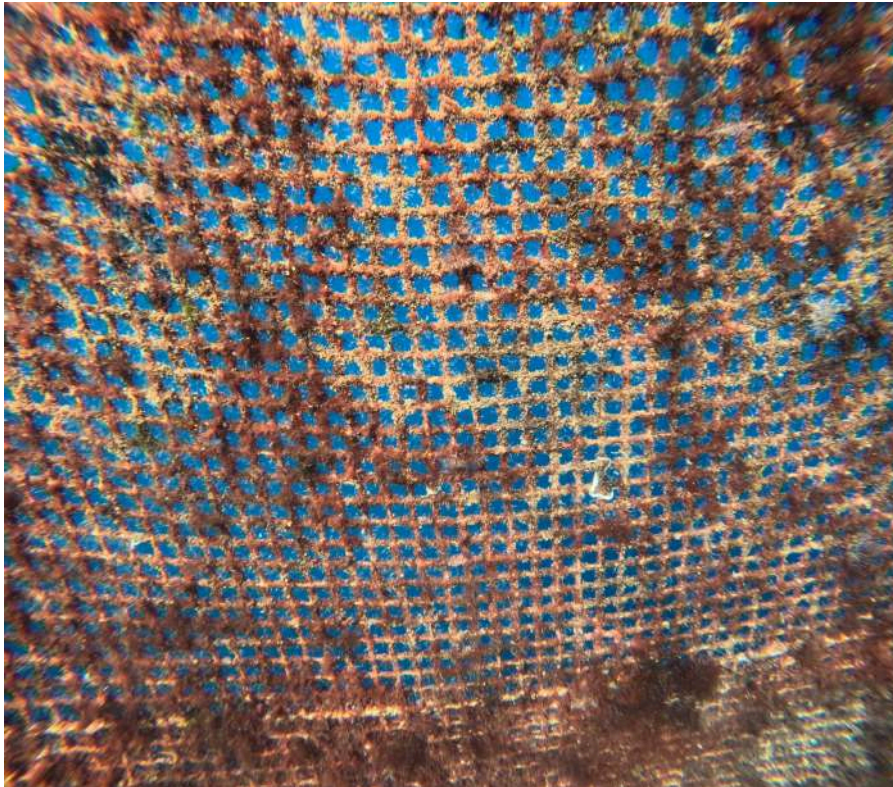


Figure 8. Photographie d'un filet recouvert de croissance marine



COURTESY OF: A. CIATTAGLIA

Figure 9. Photographie de l'application de peinture antifouling sur un filet d'aquaculture.

Par exemple, à la ferme des sanguinaires il a été remarqué après des prises de poids moyens sur des alevins de dorades que des lots âgés pourtant de 2 mois d'écarts avaient le même poids moyen. Cet écart pourrait s'expliquer selon Mr Boissier, ingénieur sur le site, par un manque d'oxygène dans la cage des alevins les plus âgés, au cours de leur croissance car les filets étaient largement encrassés. Cet incident est donc à l'origine de 2 mois de retard dans la croissance de ce lot malgré 2 mois de nourrissage supplémentaire. De la même façon des épisodes de fortes mortalités ont pu être observés à la ferme des Sanguinaires dû à des filets trop encrassés, presque 1000 par jours pour certaines cages d'alevins.

Enfin, le stress dû à la surpopulation causée par la réduction du volume des cages est également un facteur contribuant aux problèmes de santé des poissons.

Le suivi de la salissure des filets est donc un enjeu majeur pour la ferme. Des veilles sanitaires sont effectuées une fois par semaine, au cours desquelles des plongeurs inspectent les filets. Durant ces inspections, ils vérifient la présence de trous, surveillent la mortalité, et évaluent l'état de salissure des filets selon un code établi. Ce code est basé sur une échelle de 1 à 5, où 0 correspond à un filet neuf et 5 à un filet presque entièrement obstrué. Pour les filets à petites mailles (6, 8, 10 mm), l'échelle s'arrête à 3. Généralement, un filet est remplacé lorsqu'il atteint la note de 3.

Les filets encrassés sont ensuite placés dans une machine à laver géante, puis séchés avant d'être réutilisés. Cependant, des lavages trop fréquents peuvent entraîner des déformations du filet, une réduction de sa taille, et une vulnérabilité accrue à la traction. Pour ralentir le processus d'encrassement et espacer les lavages, une peinture spécifique chargée en cuivre est appliquée sur les filets.

L'article "Fishing net health state estimation using underwater imaging" de Wenliang Qiu, Vikram Pakrashi et Bidisha Ghosh qui propose une méthode d'estimation de l'état de santé des filets de pêche sous-marine, qui permet d'analyser automatiquement le degré d'encrassement des filets grâce à l'analyse d'images sous-marines, et de calculer un pourcentage de blocage pour chaque ouverture de filet. La méthode qu'ils ont nommé NHSE pour Net Health State Estimation, comporte six étapes, à savoir l'interaction avec l'utilisateur, la correction de la distorsion, le "débrumage" de l'image sous-marine, la segmentation de l'encrassement biologique, l'analyse de la structure de l'ouverture du filet et l'estimation du pourcentage d'obstruction. La seconde partie de l'article s'occupe de vérifier l'universalité et la robustesse de l'algorithme. Ils ont alors testé leur algorithme sur des images sous-marines

dans la baie de Mulroy, en Irlande. Puis, ils ont simulé et construit une ferme de pêche virtuelle et, sur cette base, testé l'algorithme sur des images de filets de pêche dans différentes conditions environnementales.

L'idée est alors de reprendre les explications des 6 étapes de l'algorithme afin de mettre au point un algorithme en langage python effectuant les mêmes fonctions que celles précisées dans l'article, puis de tester cet algorithme grâce aux photos fournies par les plongeurs.

II- L'ÉLABORATION DE L'ALGORITHME

La figure 10 illustre schématiquement, les étapes suivies par la méthode NHSE qui à partir d'une photo de filet donne une estimation du pourcentage du filet bloqué par la salissure biologique. On retrouve alors les étapes suivantes:

Entrée: Une photo d'un filet avec n'importe quel cadrage.

- 1: Sélection par l'utilisateur de la partie de l'image qui l'intéresse
- 2: Correction de la distorsion- Recadrage à plat de la portion sélectionnée du filet.
- 3: Le "débrumage": L'image initiale étant partiellement gênée par l'environnement marin alentour, on "débrumme" l'image afin d'augmenter les contrastes et parvenir à une image plus nette, aux couleurs plus discernables.
- 4: La segmentation de l'encrassement biologique - On obtient une image avec seulement 3 couleurs différentes permettant de mieux discerner ce qu'est le filet, la salissure et les ouvertures.
- 5: L'analyse de la structure du filet - détection des mailles initiales à partir de l'image segmentée.
- 6: estimation du pourcentage du filet recouvert grâce à la comparaison entre l'image segmentée et l'image des mailles donnée en 5.

Sortie: On obtient une image de la portion sélectionnée avec dans chaque maille le recouvrement de celle-ci ainsi que la moyenne de tous ces pourcentage donnant une estimation du recouvrement de la portion du filet choisie.

Pour parvenir à créer un tel algorithme j'ai utilisé tout particulièrement OpenCV

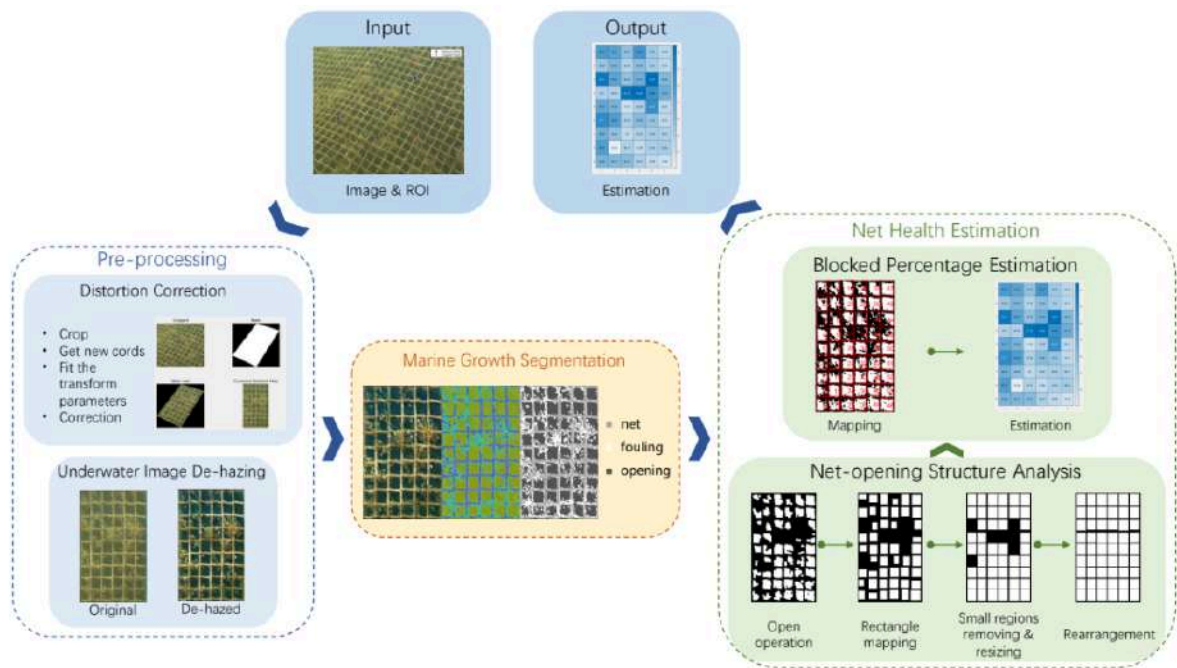


Figure 10. Schéma de la underwater fishing net health state estimation (NHSE)(Qiu et al., 2020)

Il s'agit d'une bibliothèque de traitement d'images et de vision par ordinateur très puissante et largement utilisée en Python. Je l'ai utilisée dans mon code pour ses nombreuses fonctionnalités qui facilitent la manipulation d'images, comme la correction de perspective, la segmentation, l'érosion, et la dilatation. Grâce à OpenCV, j'ai pu traiter efficacement les images du filet pour détecter et analyser les ouvertures, en automatisant des tâches complexes de manière rapide et précise.

1. Sélection par l'utilisateur de la partie de l'image qui l'intéresse

L'algorithme commence par une étape cruciale : permettre à l'utilisateur de sélectionner une portion spécifique du filet à analyser. Cette étape est réalisée grâce à une partie de la fonction appelée `correct_distortion` (fonction qui sert à l'étape suivante). L'algorithme commence par afficher l'image du filet que l'on souhaite analyser. Ensuite, l'utilisateur est invité à sélectionner quatre points sur l'image, correspondant aux quatre coins de la portion de filet qu'il souhaite analyser.

2. Correction de la distorsion

Après que les quatre coins ont été sélectionnés, l'algorithme utilise ces points pour corriger la distorsion de l'image. Cela signifie qu'il transforme la région sélectionnée en une vue "aplatie", où la portion du filet apparaît sans perspective ni distorsion.

1. Conversion des Points en Coordonnées Flottantes

Les quatre points que l'utilisateur a sélectionnés sont d'abord convertis en un tableau `numpy` de type `float32`. Cette conversion est nécessaire car la fonction de transformation de perspective utilisée plus tard attend des coordonnées en virgule flottante pour effectuer les calculs avec précision.

2. Calcul de la Largeur et de la Hauteur de la Région Sélectionnée

Le calcul de la largeur : La largeur de la région sélectionnée est calculée en prenant la distance entre les points opposés horizontalement :

- `source[0] - source[1]` : Distance entre les deux premiers points sélectionnés.

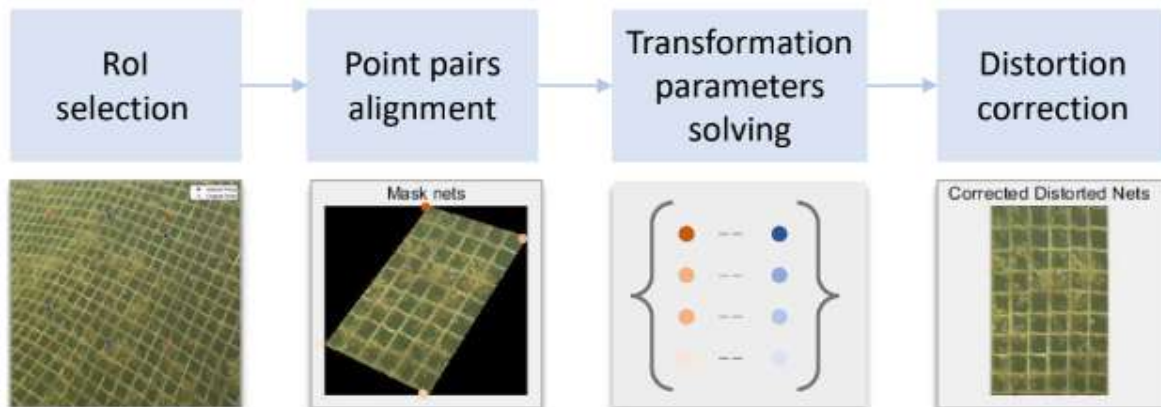


Figure 11. Schéma de la correction de la distorsion (Qiu et al., 2020)

```
def correct_distortion(image):
    global points
    points = []

    def on_key(event):
        if event.key == 'enter':
            plt.close()

    fig, ax = plt.subplots()
    ax.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
    fig.suptitle("Sélectionnez 4 coins (Appuyez sur 'entrer' quand c'est fait)")
    fig.canvas.mpl_connect('button_press_event', onclick)
    fig.canvas.mpl_connect('key_press_event', on_key)
    plt.show()

    if len(points) != 4:
        print(f"Points sélectionnés: {len(points)}")
        raise ValueError("Quatre points doivent être sélectionnés.")

    1 source = np.float32(points)
    2 width = max(np.linalg.norm(source[0] - source[1]), np.linalg.norm(source[2] - source[3]))
    height = max(np.linalg.norm(source[0] - source[2]), np.linalg.norm(source[1] - source[3]))
    3 destination = np.float32([[0, 0], [width, 0], [0, height], [width, height]])
    4 matrix = cv2.getPerspectiveTransform(source, destination)
    corrected_image = cv2.warpPerspective(image, matrix, (int(width), int(height)))

    plt.imshow(cv2.cvtColor(corrected_image, cv2.COLOR_BGR2RGB))
    plt.title('Image sélectionnée')
    plt.axis('off')
    plt.show()

    return corrected_image
```

Sélection par l'utilisateur de la partie de l'image qui l'intéresse

Affichage de l'image corrigée

Figure 12. Code de la fonction `correct\_distortion`. Les numéros correspondent aux numéros des sous parties du 2. **Corrections de la distorsion**

- `'source[2] - source[3]'` : Distance entre les deux autres points.
- La plus grande de ces deux distances est choisie comme largeur de la région. Cela permet de prendre en compte les éventuelles différences de longueur dues à la perspective.

Le calcul de la hauteur : De manière similaire, la hauteur est calculée en prenant la distance entre les points opposés verticalement :

- `'source[0] - source[2]'` : Distance entre le premier et le troisième point.
- `'source[1] - source[3]'` : Distance entre le deuxième et le quatrième point.
- Encore une fois, la plus grande de ces deux distances est choisie comme hauteur.

3. Définition des Points de Destination

Ici, on définit un nouveau jeu de quatre points, appelé `'destination'`, qui correspond à la région sélectionnée, mais redressée en une forme rectangulaire. Ces points sont définis comme cela :

- `'[0, 0]'` : Le coin supérieur gauche du rectangle.
- `'[width, 0]'` : Le coin supérieur droit.
- `'[0, height]'` : Le coin inférieur gauche.
- `'[width, height]'` : Le coin inférieur droit.

Ces points représentent les nouvelles coordonnées dans l'image rectifiée, où la portion sélectionnée apparaîtra comme un rectangle bien aligné, sans distorsion.

4. Création de la Matrice de Transformation

La fonction `'cv2.getPerspectiveTransform'` d'OpenCV calcule une matrice de transformation perspective. Cette matrice est calculée en fonction des points d'origine (`'source'`) et des points de destination (`'destination'`). Cette matrice définit comment chaque point de la région sélectionnée doit être transformé pour passer de sa position initiale à la position rectifiée dans l'image finale.

Le Résultat : Le résultat est une image (`'corrected_image'`) où la portion sélectionnée apparaît redressée, comme si elle avait été vue de face, sans distorsion due à l'angle de prise de vue initial.

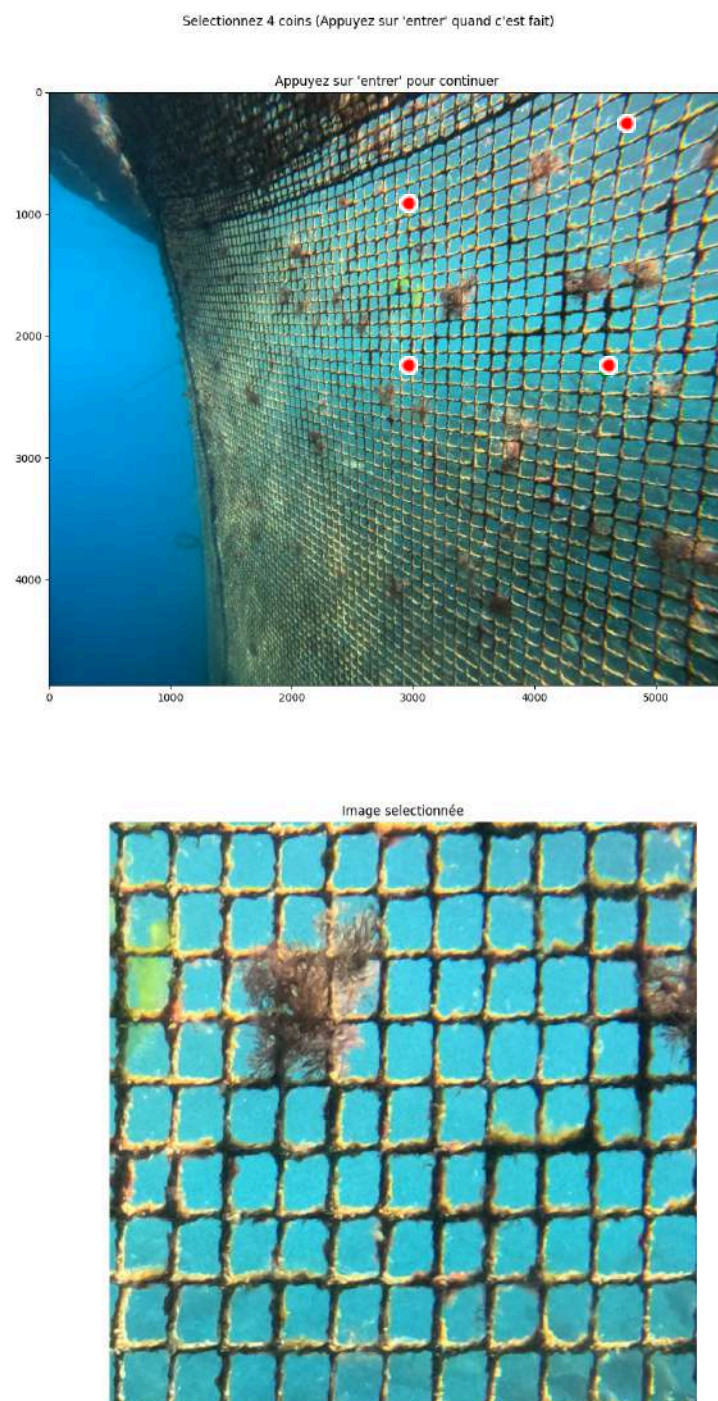


Figure 13. Résultat de la correction de distorsion sur une photo de filet de la ferme

3. Le “débrumage”

Le “débrumage” de l'image, ou "dehazing" en anglais, est une technique utilisée pour améliorer la clarté d'une image en réduisant les effets indésirables causés par la brume, le brouillard, ou d'autres conditions atmosphériques. Cette technique est particulièrement utile pour rendre une image plus nette et améliorer la visibilité des détails.

L'image d'origine, qui est dans l'espace de couleur BGR (Bleu, Vert, Rouge) utilisé par OpenCV, est convertie en espace de couleur HSV (Hue, Saturation, Value). Cet espace de couleur est souvent utilisé en traitement d'image car il sépare l'information de chrominance (couleur) de l'information de luminance (éclairage ou intensité).

L'image HSV est décomposée en trois canaux distincts :

- 'h' (Hue) : Représente la teinte ou la couleur dominante.
- 's' (Saturation) : Représente l'intensité de la couleur.
- 'v' (Value) : Représente la luminosité ou la clarté de l'image.

Le canal 'v' est particulièrement important pour le débrumage car il contient l'information sur la luminosité, qui est souvent affectée par la brume.

1. Modification du canal de luminosité ('v')

Ici, le canal 'v' est modifié pour améliorer la clarté de l'image.

- $\text{lim} = 255 - 60$: Un seuil est défini à '195' pour distinguer les zones très lumineuses des autres.
- $v[v > \text{lim}] = 255$: Les pixels du canal 'v' qui ont une valeur supérieure à '195' sont saturés à '255', ce qui signifie qu'ils deviennent complètement blancs. Cela aide à renforcer les zones déjà lumineuses.
- $v[v \leq \text{lim}] += 60$: Les autres pixels voient leur luminosité augmentée de '60' unités, ce qui améliore la clarté des zones moins lumineuses. Cela contribue à rendre les détails plus visibles en "perçant" la brume.

```

# Fonction pour débrumer l'image
def dehaze_image(image):
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
    h, s, v = cv2.split(hsv)
    1 lim = 255 - 60
      v[v > lim] = 255
      v[v <= lim] += 60
    2 final_hsv = cv2.merge((h, s, v))
      dehazed_image = cv2.cvtColor(final_hsv, cv2.COLOR_HSV2BGR)

    plt.imshow(cv2.cvtColor(dehazed_image, cv2.COLOR_BGR2RGB))
    plt.title('Image débrumée')
    plt.axis('off')
    plt.savefig('image_débrumée.png')
    plt.close()
    return dehazed_image

```

Affichage de l'image débrumée

Figure 14. Code de la fonction `dehaze\_image`. Les numéros correspondent aux numéros des sous parties du 3. Le débrumage

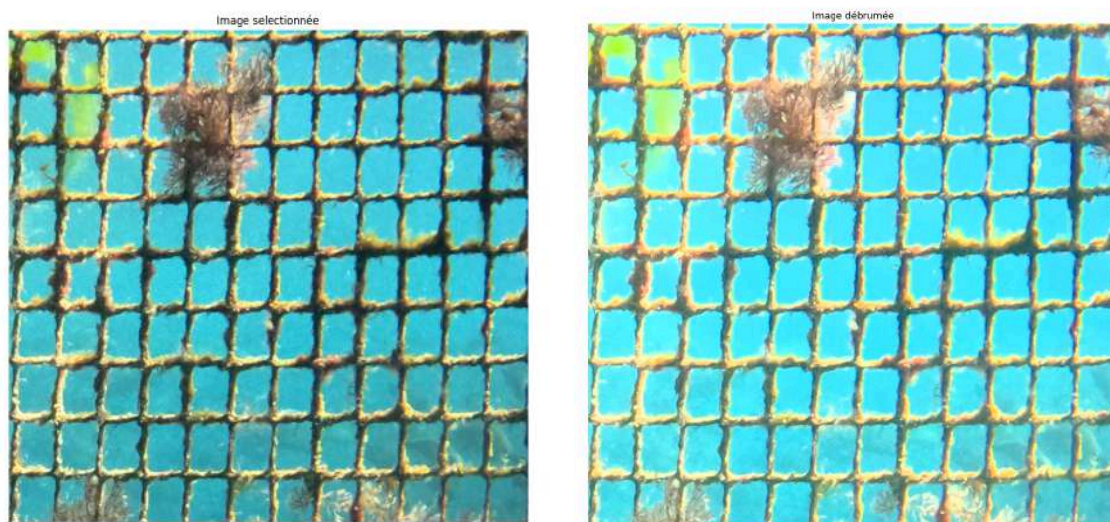


Figure 15. Résultat du débrumage

2. Reconstruction de l'image avec les modifications

Les canaux `h`, `s`, et le canal `v` modifié sont ensuite fusionnés pour reformer une image en espace HSV avec la nouvelle information de luminosité.

Puis l'image HSV modifiée est ensuite reconvertie en espace BGR, ce qui est le format standard pour les images en OpenCV. Le résultat est une image débrumée, plus claire et avec des détails plus visibles.

4. La segmentation de l'encrassement biologique.

La segmentation d'une image est un processus qui consiste à diviser une image en plusieurs segments ou régions afin de simplifier son analyse. Dans ce cas, l'objectif est de segmenter l'image pour identifier et isoler différentes parties qui pourraient correspondre à des zones spécifiques de l'image, comme des objets ou des textures. Cette segmentation aide à isoler les différentes parties de l'image (couleur du filet, des salissures et des ouvertures), ce qui facilite l'analyse ultérieure de la structure du filet.

1. Initialisation de l'algorithme

- L'image est d'abord transformée en un tableau à deux dimensions, où chaque ligne représente un pixel de l'image, et chaque colonne représente les trois canaux de couleur (Bleu, Vert, Rouge). Le résultat est un tableau de taille `(nombre\_de\_pixels, 3)`.
- Les valeurs des pixels sont ensuite converties en type `float32`. Cette conversion est importante car de nombreux algorithmes de traitement d'image, y compris le K-means, fonctionnent mieux avec des données en virgule flottante.

2. Application de l'algorithme de K-means

L'algorithme de K-means est un algorithme qui divise les données en `k` clusters, où chaque cluster correspond à un groupe de pixels ayant des caractéristiques similaires (ici, des couleurs similaires).

```

# Fonction pour segmenter l'image
def segment_image(image, k=3):
1 pixel_values = image.reshape((-1, 3))
  pixel_values = np.float32(pixel_values)
2 kmeans = KMeans(n_clusters=k, init='k-means++', random_state=42)
  labels = kmeans.fit_predict(pixel_values)
  centers = kmeans.cluster_centers_
3 centers = np.uint8(centers)
  segmented_image = centers[labels.flatten()]
  segmented_image = segmented_image.reshape(image.shape)

  plt.figure(figsize=(12, 6))
  plt.subplot(1, 2, 1)
  plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
  plt.title('Image débrumée')
  plt.axis('off')

  plt.subplot(1, 2, 2)
  plt.imshow(cv2.cvtColor(segmented_image, cv2.COLOR_BGR2RGB))
  plt.title('Image segmentée')
  plt.axis('off')

  plt.tight_layout()
  plt.show()

  return segmented_image, labels.reshape(image.shape[:2])

```

Affichage des images débrumées et segmentées côte à côte

Figure 16. Code de la fonction `segment\_image`. Les numéros correspondent aux numéros des sous parties du 4. **La segmentation de l'encrassement biologique.**

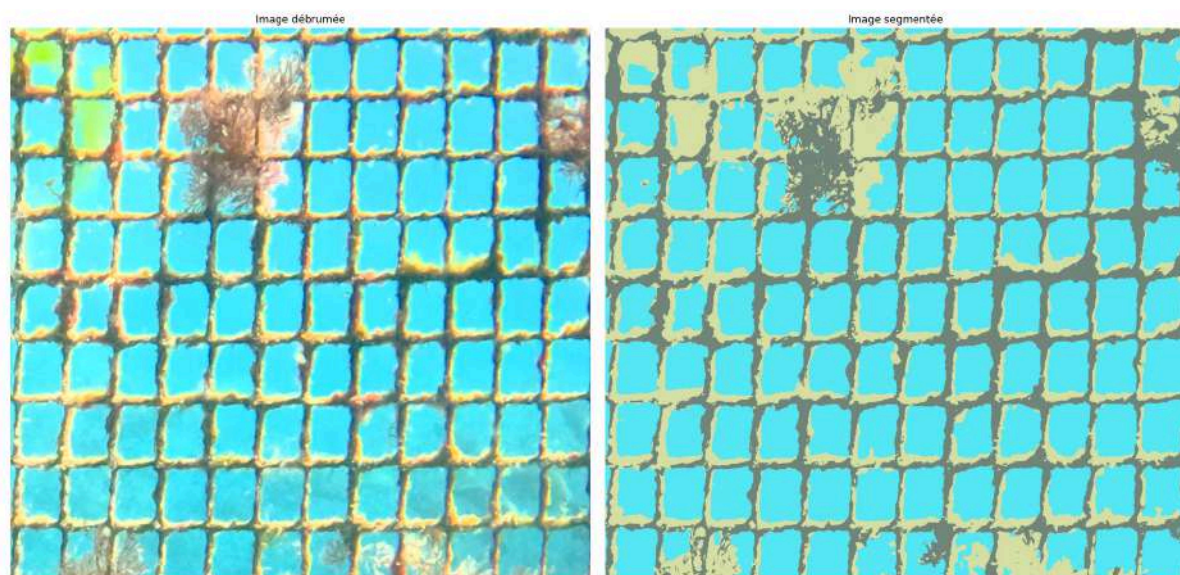


Figure 17. Résultat de la segmentation

L'option `'init=k-means++'` est utilisée pour initialiser intelligemment les centres de clusters, ce qui améliore la convergence de l'algorithme.

Le paramètre `'k'` détermine le nombre de segments ou de clusters dans lesquels l'image sera divisée. Dans ce cas, `'k=3'` signifie que l'image sera segmentée en trois régions principales. L'algorithme attribue chaque pixel au cluster le plus proche, ce qui est enregistré dans le tableau `'labels'`. Chaque élément de `'labels'` correspond à l'étiquette du cluster auquel un pixel appartient.

3. Reconstruction de l'image segmentée

Centre des clusters: Après la segmentation, les centres des clusters sont calculés. Ces centres représentent la couleur moyenne de chaque segment.

La reconstruction des pixels : L'image segmentée est reconstruite en remplaçant chaque pixel par la couleur de son centre de cluster. Cela simplifie l'image en ne gardant que `'k'` couleurs distinctes.

L'image segmentée est ensuite reformée pour avoir la même forme que l'image d'origine, c'est-à-dire une matrice 3D de taille `'(hauteur, largeur, 3)'`.

5. L'analyse de la structure du filet.

1. Extraction des canaux de couleur

La Combinaison des canaux : Le masque de l'eau est créé en combinant les canaux de couleur. L'idée ici est que l'eau ou les ouvertures dans le filet auront des caractéristiques de couleur spécifiques qui peuvent être isolées en utilisant une formule simple :

- `'(blue + green) - red'` : Cette combinaison est souvent utilisée pour détecter des zones avec un ton aquatique ou un arrière-plan où le bleu et le vert dominant, tandis que le rouge est moins prononcé.

La normalisation : Le masque est ensuite normalisé pour que ses valeurs soient comprises entre 0 et 255. Cette normalisation permet de convertir les résultats en une image de type `'uint8'` standard, facilitant les étapes suivantes.

```

def la_big_fonction(segmented_image, sr=5, MAX_X=200, MAX_Y=200):
    # Extraire les canaux de couleur et créer le masque de l'eau
    blue = resized_image[:, :, 0].astype(np.float32)
    green = resized_image[:, :, 1].astype(np.float32)
1   red = resized_image[:, :, 2].astype(np.float32)
    water_mask = (blue + green) - red
    water_mask = (water_mask - water_mask.min()) / (water_mask.max() - water_mask.min()) * 255
    water_mask = water_mask.astype(np.uint8)

2   # Seuil pour créer l'image binaire
    _, binary_image = cv2.threshold(water_mask, 240, 255, cv2.THRESH_BINARY)

3   # Appliquer l'érosion et la dilatation
    kernel = np.ones((5, 5), np.uint8)
    opening = cv2.morphologyEx(binary_image, cv2.MORPH_OPEN, kernel)

4   # Trouver les contours et créer des rectangles englobants
    contours, _ = cv2.findContours(opening, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    bounding_rectangles = []

5   for contour in contours:
        x, y, w, h = cv2.boundingRect(contour)
        bounding_rectangles.append((x, y, w, h))

6   # Ajuster les rectangles
    adjusted_rectangles = adjust_rectangles(bounding_rectangles, sr * scale_factor, MAX_X * scale_factor, MAX_Y * scale_factor)

```

Figure 18. Code de la fonction `la\_big\_fonction`. Les numéros correspondent aux numéros des sous parties du 5. **L'analyse de la structure du filet.**

2. Seuil pour créer l'image binaire

Un seuil est tout d'abord appliqué sur le masque de l'eau pour créer une image binaire :

- Les pixels ayant une valeur supérieure à 240 (ce qui correspond à des zones très claires, potentiellement les ouvertures du filet) sont transformés en blanc (valeur 255).
- Les autres pixels sont transformés en noir (valeur 0).

Enfin le résultat est une image binaire où les zones correspondant potentiellement aux ouvertures du filet apparaissent en blanc, et le reste de l'image en noir. Cette image binaire est utilisée pour identifier et analyser les ouvertures du filet dans les étapes suivantes de la fonction.

3. L'érosion et de la dilatation

Ces opérations sont utilisées pour nettoyer l'image binaire obtenue précédemment, en éliminant le bruit (petites imperfections) et en connectant les régions disjointes qui devraient être ensemble.

Un noyau de taille 5x5 est créé. Le noyau est une matrice de pixels qui détermine la forme de l'opération morphologique. Ici, un noyau carré est utilisé, ce qui est assez courant pour ces types d'opérations.

L'érosion réduit les régions blanches dans l'image binaire en « rongant » les bords des objets blancs. Cela a pour effet de supprimer les petites zones blanches isolées, qui sont souvent du bruit. Après l'érosion, la dilatation est appliquée. Elle agrandit les régions blanches restantes, en restaurant et en consolidant les zones d'intérêt principales qui ont été légèrement réduites par l'érosion. La combinaison des deux opérations est souvent appelée « ouverture » (`cv2.MORPH_OPEN`), qui est particulièrement utile pour nettoyer les images tout en conservant les structures principales.

Le résultat est une image binaire propre (`opening`), où les ouvertures du filet sont mieux définies, et le bruit est significativement réduit.

```

def la_big_fonction(segmented_image, sr=5, MAX_X=200, MAX_Y=200):
    # Extraire les canaux de couleur et créer le masque de l'eau
    blue = resized_image[:, :, 0].astype(np.float32)
    green = resized_image[:, :, 1].astype(np.float32)
    1 red = resized_image[:, :, 2].astype(np.float32)
    water_mask = (blue + green) - red
    water_mask = (water_mask - water_mask.min()) / (water_mask.max() - water_mask.min()) * 255
    water_mask = water_mask.astype(np.uint8)

    2 # Seuil pour créer l'image binaire
    _, binary_image = cv2.threshold(water_mask, 240, 255, cv2.THRESH_BINARY)

    3 # Appliquer l'érosion et la dilatation
    kernel = np.ones((5, 5), np.uint8)
    opening = cv2.morphologyEx(binary_image, cv2.MORPH_OPEN, kernel)

    4 # Trouver les contours et créer des rectangles englobants
    contours, _ = cv2.findContours(opening, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    bounding_rectangles = []

    5 for contour in contours:
        x, y, w, h = cv2.boundingRect(contour)
        bounding_rectangles.append((x, y, w, h))

    6 # Ajuster les rectangles
    adjusted_rectangles = adjust_rectangles(bounding_rectangles, sr * scale_factor, MAX_X * scale_factor, MAX_Y * scale_factor)

```

Figure 18. Code de la fonction 'la_big_fonction'. Les numéros correspondent aux numéros des sous parties du 5. **L'analyse de la structure du filet.**

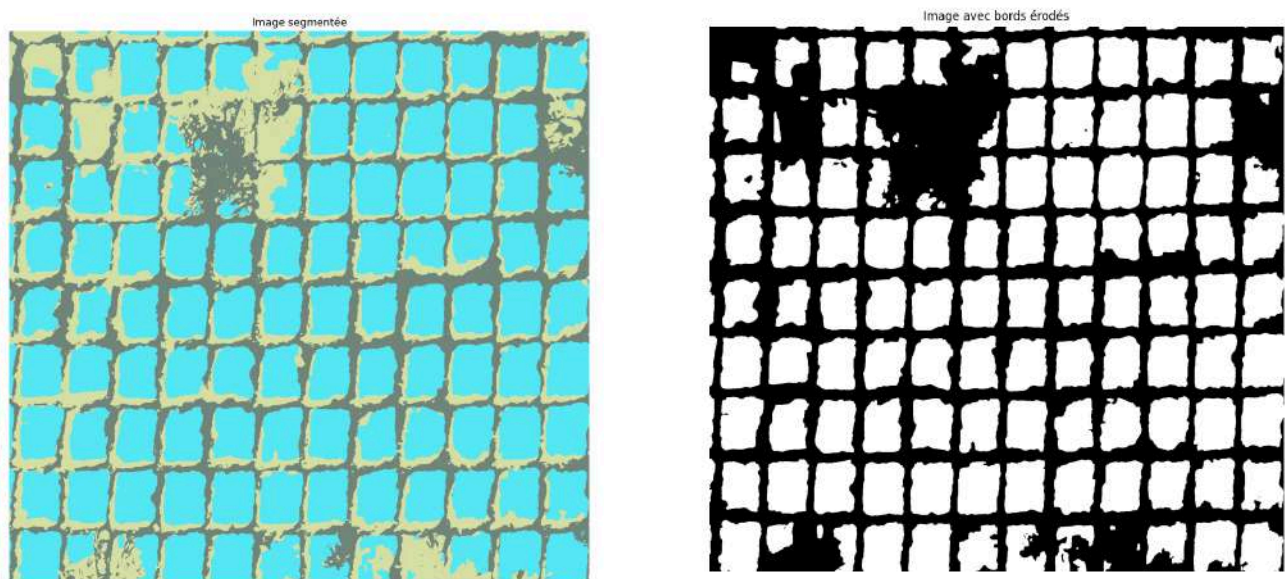


Figure 19. Résultat de la première partie de 'la_big_fonction': la création du masque de l'eau.

4. Trouver les contours

La détection des contours:

Les contours sont les limites ou les bords des objets blancs dans l'image binaire.

`cv2.findContours` est utilisé pour détecter ces contours.

- Le mode de récupération (`cv2.RETR\_EXTERNAL`) : Seuls les contours externes sont récupérés, ce qui signifie que l'on ne cherche que les bords extérieurs des objets, pas les détails internes.

- La Méthode d'approximation (`cv2.CHAIN\_APPROX\_SIMPLE`) : Cette méthode simplifie les contours en ne gardant que les points essentiels qui définissent les formes, réduisant ainsi la complexité des données.

5. Création des rectangles englobants:

- Pour chaque contour détecté, un rectangle englobant est créé à l'aide de la fonction `cv2.boundingRect`.

- `boundingRect(contour)` : Cette fonction retourne un rectangle minimal (défini par son coin supérieur gauche `(x, y)` et sa largeur `w` et hauteur `h`) qui peut contenir tout le contour.

- La liste des rectangles : Les coordonnées et les dimensions de ces rectangles sont stockées dans une liste `bounding\_rectangles`.

Ces rectangles représentent les premiers "cadrages" des ouvertures du filet dans l'image. Ces rectangles ne sont pas encore ajustés de manière optimale, mais ils servent de base pour l'étape suivante, où l'ajustement et l'optimisation des rectangles seront effectués pour une analyse plus précise.

6. L'ajustement des rectangles

L'ajustement des rectangles est réalisé dans la fonction `adjust\_rectangles`, qui est appelée à partir de `la\_big\_fonction`. L'article explique la méthode permettant de modéliser le maillage initial à l'aide de l'algorithme décrit dans la [figure 21](#). Cependant, en essayant de suivre cette méthode, je n'ai pas obtenu de résultats satisfaisants ; le maillage obtenu ne correspondait pas à l'apparence que devrait avoir le filet en l'absence de biofouling. Il est possible que la

```

def la_big_fonction(segmented_image, sr=5, MAX_X=200, MAX_Y=200):
    # Extraire les canaux de couleur et créer le masque de l'eau
    blue = resized_image[:, :, 0].astype(np.float32)
    green = resized_image[:, :, 1].astype(np.float32)
    red = resized_image[:, :, 2].astype(np.float32)
    water_mask = (blue + green) - red
    water_mask = (water_mask - water_mask.min()) / (water_mask.max() - water_mask.min()) * 255
    water_mask = water_mask.astype(np.uint8)

    # Seuil pour créer l'image binaire
    _, binary_image = cv2.threshold(water_mask, 240, 255, cv2.THRESH_BINARY)

    # Appliquer l'érosion et la dilatation
    kernel = np.ones((5, 5), np.uint8)
    opening = cv2.morphologyEx(binary_image, cv2.MORPH_OPEN, kernel)

    # Trouver les contours et créer des rectangles englobants
    contours, _ = cv2.findContours(opening, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    bounding_rectangles = []

    for contour in contours:
        x, y, w, h = cv2.boundingRect(contour)
        bounding_rectangles.append((x, y, w, h))

    # Ajuster les rectangles
    adjusted_rectangles = adjust_rectangles(bounding_rectangles, sr * scale_factor, MAX_X * scale_factor, MAX_Y * scale_factor)

```

Figure 18. Code de la fonction 'la_big_fonction'. Les numéros correspondent aux numéros des sous-parties du 5. **L'analyse de la structure du filet.**

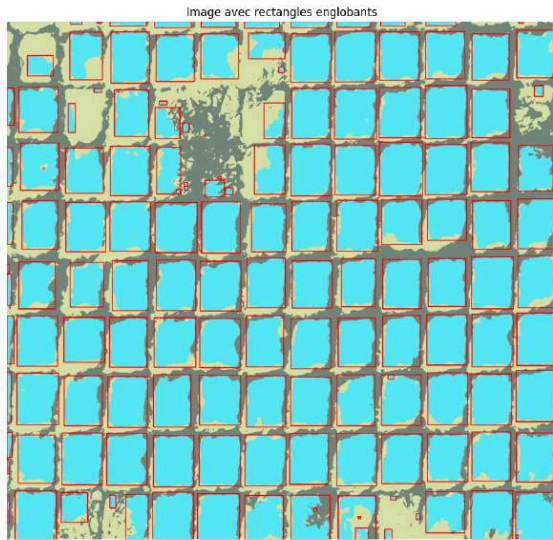


Figure 20. La création des rectangles englobants

Algorithm 2 Region resizing and Rearrangement

Input: $St = \{St_i\}$
Output: St^*

- 1: the search range: sr ;
- 2: the centre of i -th rectangle: $C_i = (x_i, y_i)$;
- 3: index of column has maximum number of regions: MAX_idx_col ;
- 4: index of row has maximum number of regions: MAX_idx_row ;
- 5: **initialization**;
- 6: **for** St_i **do**
- 7: **search in column**:
- 8: $idx_col = \{i | x_i \in [\max(x_i - sr, 0), \min(x_i + sr, MAX_X)]\}$
- 9: **if** $\#idx_col > \#MAX_idx_col$ **then**
- 10: $MAX_idx_col \leftarrow idx_col$
- 11: **search in row**:
- 12: $idx_row = \{i | y_i \in [\max(y_i - sr, 0), \min(y_i + sr, MAX_Y)]\}$
- 13: **if** $\#idx_row > \#MAX_idx_row$ **then**
- 14: $MAX_idx_row \leftarrow idx_row$
- 15: **update** $C_i \leftarrow \text{mean}(C_{idx})$; $H_i \leftarrow \text{mean}(H_{idx})$; $W_i \leftarrow \text{mean}(W_{idx})$
- 16: **for** $y_m, m \in \{MAX_idx_col\}$ **do**
- 17: **for** $x_n, n \in \{MAX_idx_row\}$ **do**
- 18: **if** $(x_i, y_i) \notin [(x_n - sr, y_m - sr), (x_n + sr, y_m + sr)]$ **then**
- 19: $W_{mean} \leftarrow \frac{1}{k} \sum_{s=1}^k W(x_s, y_m)$
- 20: $H_{mean} \leftarrow \frac{1}{l} \sum_{s=1}^l H(x_n, y_s)$
- 21: **create new** $St \{(x_i, y_i); W_{mean}; H_{mean}\} \rightarrow St^*$
- 22: **return** St^*

2.3.2. Blocked Percentage Estimation

The structure for each cell St_i is obtained by the net-opening structure analysis module. In addition, the net-opening segmentation results can be treated as the area O_i , and then the blocked percentage BP_i can be computed simply as:

$$BP_i = \frac{O_i}{St_i} \quad (10)$$

Figure 21. Algorithme de l'ajustement des premiers rectangles selon l'article "Fishing net health state estimation using underwater imaging"

méthode décrite dans l'article ne soit pas entièrement complète, ou que j'aie mal interprété certaines étapes.

J'ai donc décidé de modifier l'approche. Selon moi, le problème de leur méthode réside dans le fait qu'un rectangle est ajusté en fonction de la moyenne des rectangles déjà existants dans sa ligne et sa colonne. Son centre, sa largeur, et sa hauteur deviennent alors les moyennes respectives des centres, largeurs, et hauteurs des rectangles existants dans les mêmes indices de lignes et colonnes. Par conséquent, si la majorité des rectangles aux mêmes indices sont recouverts de salissure marine, le rectangle ajusté sera plus petit que la maille réelle non encrassée.

Pour résoudre ce problème, j'ai supposé que le plus grand rectangle englobant correspond à la maille la moins recouverte possible, c'est-à-dire la forme approximative d'une maille intacte du filet. En appliquant les dimensions de ce rectangle à tous les autres, on obtient une approximation plus fidèle. La méthode pour déterminer le centre des mailles reste celle décrite dans l'article.

L'algorithme fonctionne donc de la manière suivante :

Etape 1: Filtrage des petits rectangles

La première étape consiste à calculer l'aire ($w * h$) de chaque rectangle englobant. Ensuite, on calcule l'aire moyenne de ces rectangles. On ne conserve que les rectangles dont l'aire est supérieure ou égale à 40% de l'aire moyenne des rectangles. Les petits rectangles, souvent créés à cause du bruit ou des petites imperfections dans l'image, sont ainsi éliminés. Cela permet de ne garder que les rectangles représentant les grandes ouvertures du filet.

Etape 2: Création des indexs des rectangles et calcul des centres

Le centre de chaque rectangle est calculé en prenant la moyenne des coordonnées (x, y) de chaque coin du rectangle. Le centre est utilisé pour aligner les rectangles sur une grille régulière dans les étapes suivantes.

6

```

def adjust_rectangles(rectangles, sr, MAX_X, MAX_Y):
    # 1. Enlever les petits rectangles
    areas = [w * h for x, y, w, h in rectangles]
    mean_area = np.mean(areas)
    filtered_rectangles = [(x, y, w, h) for x, y, w, h in rectangles if w * h >= 0.4 * mean_area]

    if not filtered_rectangles:
        return []

    # 2. Création des indexs des rectangles et calcul des centres
    centers = [(x + w / 2, y + h / 2) for x, y, w, h in filtered_rectangles]

    idx_col = {i: [] for i in range(len(filtered_rectangles))}
    idx_row = {i: [] for i in range(len(filtered_rectangles))}

    for i, (cx1, cy1) in enumerate(centers):
        for j, (cx2, cy2) in enumerate(centers):
            if max(cx2 - sr, 0) <= cx1 <= min(cx2 + sr, MAX_X):
                idx_col[i].append(j)
            if max(cy2 - sr, 0) <= cy1 <= min(cy2 + sr, MAX_Y):
                idx_row[i].append(j)

    # 3. Ajuster la taille des rectangles
    max_width = max(filtered_rectangles, key=lambda r: r[2])[2]
    max_height = max(filtered_rectangles, key=lambda r: r[3])[3]

    for i, (x, y, w, h) in enumerate(filtered_rectangles):
        filtered_rectangles[i] = (x, y, max_width, max_height)

    # 4. Ajuster le centre des rectangles
    optimized_rectangles = []
    for i in range(len(filtered_rectangles)):
        x, y, w, h = filtered_rectangles[i]

        # Moyenne des centres pour les colonnes et les rangées, avec vérification
        if idx_col[i]:
            mean_cx = np.mean([centers[j][0] for j in idx_col[i]])
        else:
            mean_cx = centers[i][0]

        if idx_row[i]:
            mean_cy = np.mean([centers[j][1] for j in idx_row[i]])
        else:
            mean_cy = centers[i][1]

        # Mettre à jour le centre du rectangle en conséquence
        optimized_rectangles.append((int(mean_cx - w / 2), int(mean_cy - h / 2), w, h))

    # 5. Ajouter les rectangles manquants
    col_centers = {}
    row_centers = {}

    for i in range(len(filtered_rectangles)):
        mean_cx = np.mean([centers[j][0] for j in idx_col[i]]) if idx_col[i] else centers[i][0]
        mean_cy = np.mean([centers[j][1] for j in idx_row[i]]) if idx_row[i] else centers[i][1]

        col_centers.setdefault(mean_cx, []).append(mean_cy)
        row_centers.setdefault(mean_cy, []).append(mean_cx)

    new_rectangles = set()
    for mean_cx in col_centers:
        for mean_cy in row_centers:
            exists = any(abs(mean_cx - (x + w / 2)) < sr and abs(mean_cy - (y + h / 2)) < sr for x, y, w, h in optimized_rectangles)
            if not exists:
                Wmean = max_width
                Hmean = max_height
                new_rectangles.add((int(mean_cx - Wmean / 2), int(mean_cy - Hmean / 2), Wmean, Hmean))

    optimized_rectangles.extend(new_rectangles)

    return optimized_rectangles

```

Figure 22. Code de la fonction `adjust\_rectangles`.

Étape 3: Ajustement de la taille des rectangles et de leurs centres

Cette étape comprends:

- Le calcul des dimensions maximales : On calcule la largeur ('max_width') et la hauteur ('max_height') maximales parmi les rectangles filtrés.
- L'Uniformisation des tailles : Chaque rectangle est ensuite redimensionné pour avoir la même largeur et hauteur maximales.

Pour chaque rectangle, on aligne son centre sur la moyenne des centres des rectangles voisins dans la même colonne ('idx_col[i]') ou la même rangée ('idx_row[i]'). Si un rectangle n'a pas de voisins proches dans une colonne ou une rangée, son centre reste inchangé.

Enfin, les rectangles sont repositionnés en fonction de ces nouveaux centres calculés, garantissant une répartition des rectangles le plus proche du maillage à neuf.

Étape 4: L'ajout des rectangles manquants

Du fait que certaines mailles sont trop recouvertes par la salissure pour être reconnu comme des ouvertures par l'algorithme ou bien que l'ouverture ai été classée comme trop petite (aire du rectangle englobant inférieur ou égale à 40% de l'aire moyen des autres rectangles), Des rectangles peuvent manquer à ce stade. Ainsi on procède comme suivant:

On identifie les positions où il devrait y avoir un rectangle en fonction de la grille formée par les centres des rectangles déjà optimisés. Si un espace est vide, un nouveau rectangle de la taille standard ('max_width', 'max_height') est ajouté à cette position. Cela permet de remplir les trous dans la grille, assurant une couverture complète et régulière de l'image.

Résultat

On obtient alors une grille correspondant approximativement au maillage de la portion de filet sélectionnée si elle n'était pas recouverte de salissure marine.

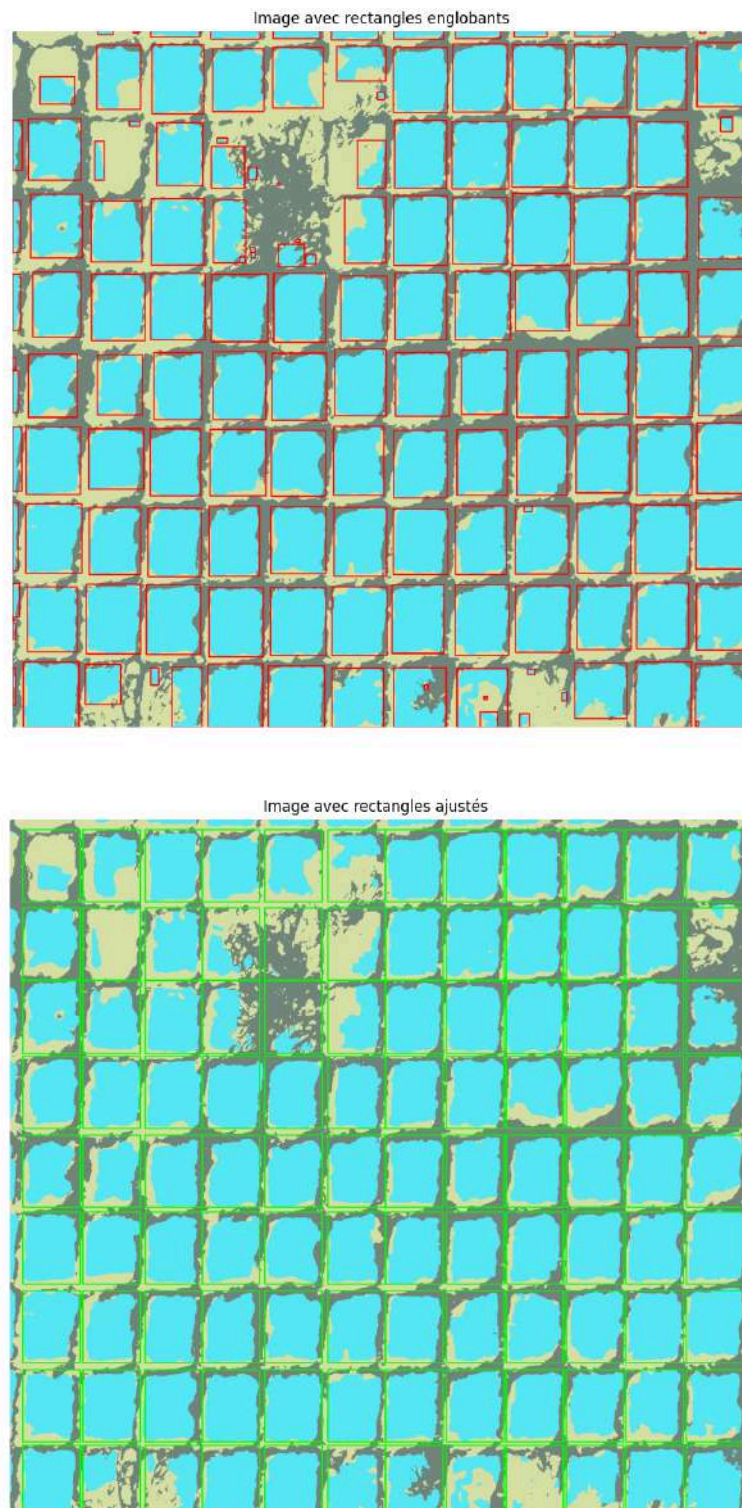


Figure 23. Résultat de l'ajustement de rectangles

6. Estimation du pourcentage du filet recouvert.

Le calcul du pourcentage de recouvrement est la finalité de cet algorithme. Il est permis grâce à la superposition de l'image binaire (qui représente en blanc les ouvertures du filet) et des rectangles ajustés donnés par 'la big fonction'.

Le calcul du pourcentage de recouvrement est effectué dans la fonction 'calculate_coverage'.

1. Boucle sur les rectangles pour calculer le pourcentage de recouvrement

La liste 'coverage_percentages' est tout d'abord initialisée pour stocker le pourcentage de recouvrement de chaque rectangle.

Puis pour chaque rectangle défini par ses coordonnées '(x, y)' et ses dimensions '(w, h)', l'algorithme procède comme suit :

- Calcul de l'aire du rectangle : L'aire du rectangle est calculée par 'rect_area = w * h'. Cela représente la zone totale du rectangle dans laquelle on va évaluer la couverture.

- Extraction de la région d'intérêt (ROI pour region of interest): L'algorithme extrait la partie correspondante de l'image binaire en utilisant les coordonnées du rectangle : 'roi = binary_image[y:y+h, x:x+w]'.

- Calcul de la zone noire (couverture) : Dans l'image binaire, les pixels noirs (valeur 0) représentent l'eau. L'algorithme compte le nombre de pixels noirs dans la région d'intérêt avec 'black_area = np.sum(roi == 0)'.

- Calcul du pourcentage de recouvrement : Le pourcentage de recouvrement pour ce rectangle est calculé en divisant la zone noire par l'aire totale du rectangle : 'coverage_percentage = (black_area / rect_area) * 100'. Ce pourcentage est ensuite ajouté à la liste 'coverage_percentages'.

2. Calcul du pourcentage de recouvrement total de la portion

Après avoir traité tous les rectangles, l'algorithme calcule la moyenne des pourcentages de recouvrement : 'mean_coverage = np.mean(coverage_percentages)'.

```
def calculate_coverage(binary_image, rectangles):
    coverage_percentages = []

    1 for x, y, w, h in rectangles:
        rect_area = w * h
        roi = binary_image[y:y+h, x:x+w]
        black_area = np.sum(roi == 0)
        coverage_percentage = (black_area / rect_area) * 100
        coverage_percentages.append(coverage_percentage)

    2 mean_coverage = np.mean(coverage_percentages)
    return coverage_percentages, mean_coverage
```

Figure 24. Code de la fonction `calculate\_coverage`. Les numéros correspondent aux numéros des sous parties du 6. **Estimation du pourcentage du filet recouvert.**

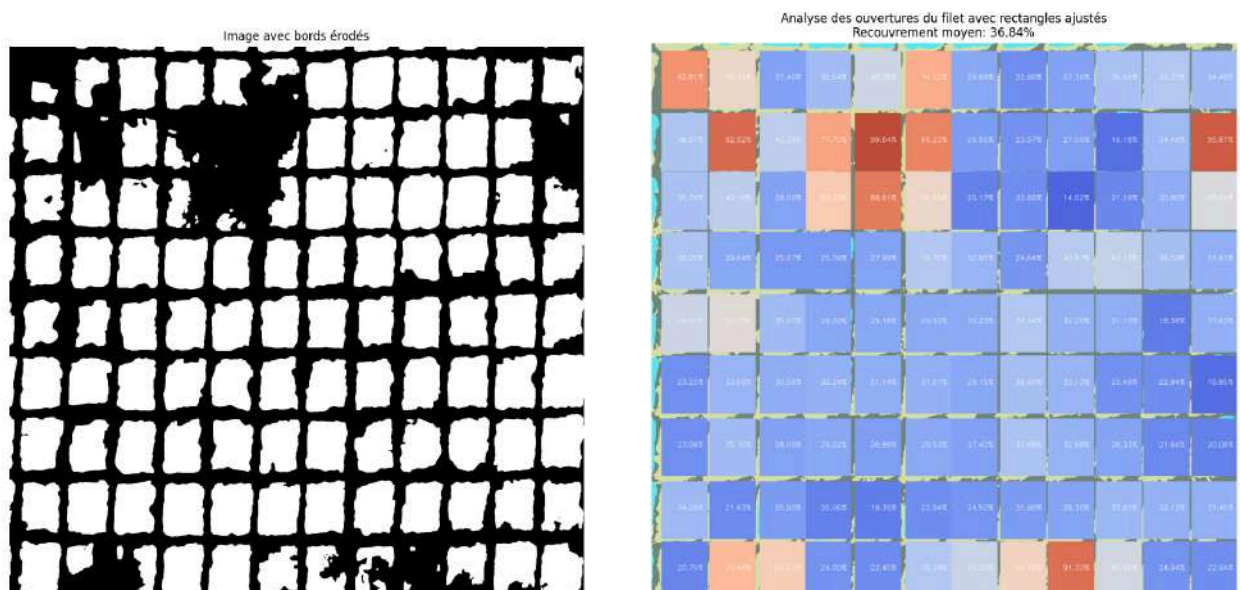


Figure 25. Résultat du calcul du pourcentage de recouvrement de la portion de filet initialement choisie

La fonction retourne la liste des pourcentages de recouvrement pour chaque rectangle ainsi que le pourcentage moyen de recouvrement.

Une dernière fonction est ajoutée à l'algorithme afin de créer un fichier word récapitulant les principaux résultats de l'analyse: Une photo de la portion sélectionnée du filet, l'image binarisée ressortant et l'image finale représentant le maillage découvert.

III-ANALYSE DES RÉSULTATS DE L'ALGORITHME SUR DIVERS FILETS

Après avoir détaillé le fonctionnement de l'algorithme de détection et d'analyse des mailles des filets, cette section se concentre sur les résultats obtenus à partir de son application sur une série de tests en conditions réelles. Ces tests ont été réalisés sur des filets de différentes tailles de mailles, constitués de matériaux variés tels que le nylon et le dyneema, afin de garantir la robustesse et la polyvalence de l'algorithme dans des contextes variés.

1.Explication de la démarche

Les photos utilisées pour ces tests ont été capturées à l'aide d'une caméra GoPro 10, montée sur une perche de 2 mètres. La caméra était immergée à une profondeur approximative de 2 mètres sous l'eau, depuis l'intérieur de l'une des cages. Les prises de vue ont été effectuées dans des conditions diverses, incluant des variations d'orientation, d'angle, d'exposition, et de distance par rapport au filet. Pour chaque cage, trois photos ont été capturées : deux depuis l'extérieur et une depuis l'intérieur de la cage, permettant ainsi une comparaison entre le niveau d'encrassement perçu de l'extérieur et celui observé depuis l'intérieur, en présence des poissons.

Les 48 photos utilisées dans cette étude proviennent de 16 cages différentes, à raison de trois photos par cage. Les 11 premières cages ont été choisies en suivant mon parcours habituel pour la gestion des filets de rétention, filets destinés à collecter les œufs potentiels de divers parasites affectant les dorades et les maigres.

Ces filets sont régulièrement remontés pour sécher pendant deux jours, ce qui élimine les œufs, avant d'être descendus à nouveau. Les cinq cages restantes, choisies en raison de leurs mailles plus grandes, ont été sélectionnées pour représenter une plus grande diversité de conditions, incluant des niveaux de salissures variés.

Cette diversité dans les conditions de prise de vue et les types de filets permet de tester l'algorithme de manière exhaustive, en évaluant sa capacité à détecter et à analyser efficacement les mailles des filets dans des environnements et des situations variés.

2. Analyse des Résultats : Adaptabilité et Limites de l'Algorithme

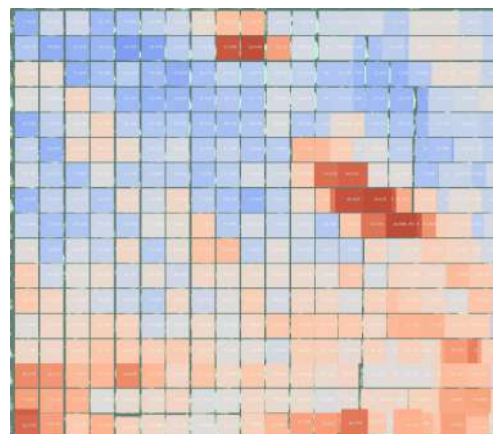
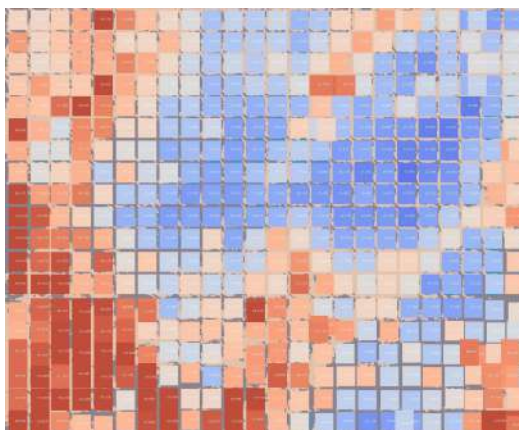
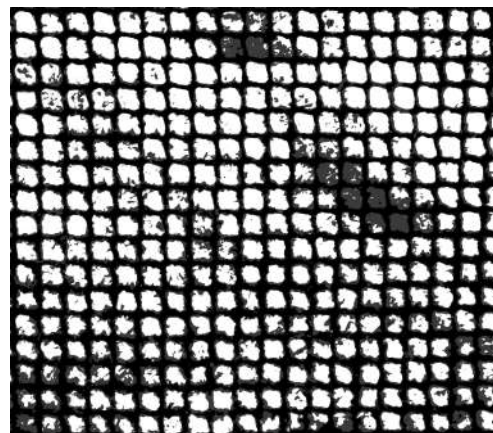
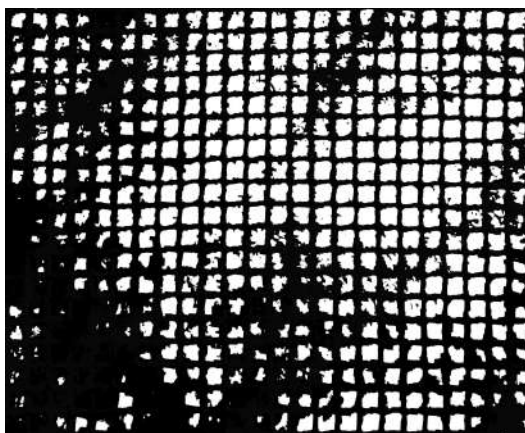
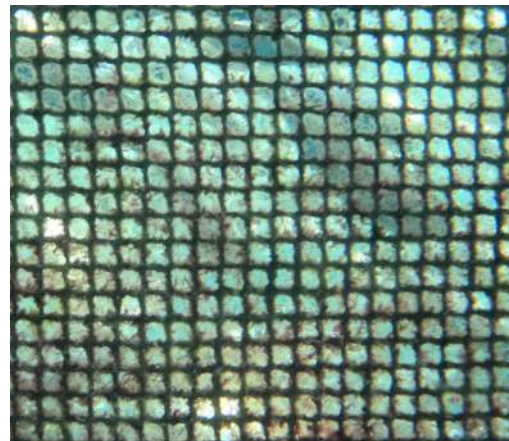
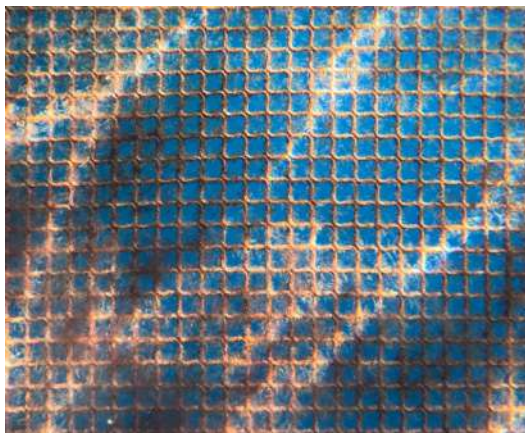
L'application de l'algorithme de détection des ouvertures des filets dans diverses conditions a révélé à la fois sa robustesse et certaines de ses limitations. Voici une analyse détaillée des résultats obtenus, en tenant compte des différents défis rencontrés lors des tests sur le terrain.

Adaptation aux reflets

L'algorithme a montré une bonne adaptabilité face à la présence de reflets sur l'eau. Comme les reflets proviennent généralement du filet lui-même, mais avec une teinte plus claire, cela n'a pas entravé la détection des ouvertures. En effet, la composante bleue de l'image, qui est cruciale pour la détection des mailles, n'est pas significativement altérée par ces reflets, permettant ainsi à l'algorithme de continuer à fonctionner correctement sans confondre les reflets avec des ouvertures ou des zones d'encrassement.

Impact des éléments derrière le filet

Un problème majeur rencontré concerne la présence de poissons, notamment lorsqu'ils sont très proches du filet. L'algorithme tend à détecter ces poissons comme des salissures, ce qui fausse l'analyse. Ce phénomène est particulièrement prononcé dans les cages à forte densité, telles que celles contenant des alevins. Par conséquent, il est recommandé de privilégier les prises de vue à l'intérieur des cages, où l'on évite ce genre d'interférence. Cette approche permet une meilleure précision dans la détection des salissures réelles du filet, en éliminant les distractions causées par la faune présente derrière le filet.



A

B

Figure 26. Deux résultats de traitement obtenus par l'algorithme. **A:** l'adaptation à la présence de reflets sur la photographie: les parties éclairées mettant plus en avant le recouvrement de la zone, l'algorithme les perçoit plus facilement, cependant on note que les parties couvertes et ombragées sont tout de même reconnues. **B:** Impact des éléments derrière le filet: les poissons, ici des alevins de dorades, sont reconnus comme du biofouling

Limites du Postulat de Base sur les Mailles Propres

L'algorithme repose sur l'hypothèse qu'au moins une des mailles sélectionnées est sans encrassement, permettant de définir un seuil de référence pour l'analyse. Cependant, cette hypothèse se révèle problématique sur des filets très encrassés. Dans ces cas-là, la maille avec la plus grande ouverture détectée peut être elle-même partiellement obstruée par des salissures, conduisant à une sous-estimation du pourcentage de recouvrement. Ce biais peut fausser l'interprétation des résultats, suggérant une propreté relative du filet alors que l'encrassement est en réalité plus généralisé.

Précision dans la Détection des Courbes du Filet

L'algorithme s'adapte bien aux courbes naturelles du filet, détectant les ouvertures avec précision même sur des filets qui ne sont pas parfaitement tendus. Cependant, lorsque le filet est trop déformé, la répartition des rectangles dans les colonnes et les lignes devient inexacte, ce qui affecte la qualité de l'analyse. Cette limitation est particulièrement perceptible sur des filets très déformés, où les mailles ne suivent plus une grille régulière, rendant difficile la tâche de l'algorithme pour segmenter correctement l'image.

Indifférence aux couleurs des salissures

Un aspect positif de l'algorithme est son indifférence relative aux couleurs des salissures biologiques. Que les algues soient vertes, rouges ou d'autres teintes, cela n'affecte pas significativement l'analyse. L'algorithme se concentre principalement sur les objets présentant une composante bleue plus prononcée, ce qui lui permet de se focaliser efficacement sur les ouvertures du filet, indépendamment de la couleur des salissures présentes.

Problèmes avec la variation de couleur de l'eau

La variation de la couleur de l'eau sur une même photo pose cependant un défi. L'eau peut s'assombrir avec la profondeur, ce qui peut créer des gradients de couleur du haut vers le bas de l'image. Si cette zone d'eau sombre est minoritaire sur la photo, elle risque d'être confondue avec des salissures, perturbant ainsi l'analyse. Ce problème survient particulièrement lorsque l'algorithme traite des images où la variation de couleur de l'eau est significative, mais n'occupe qu'une petite portion de l'image.

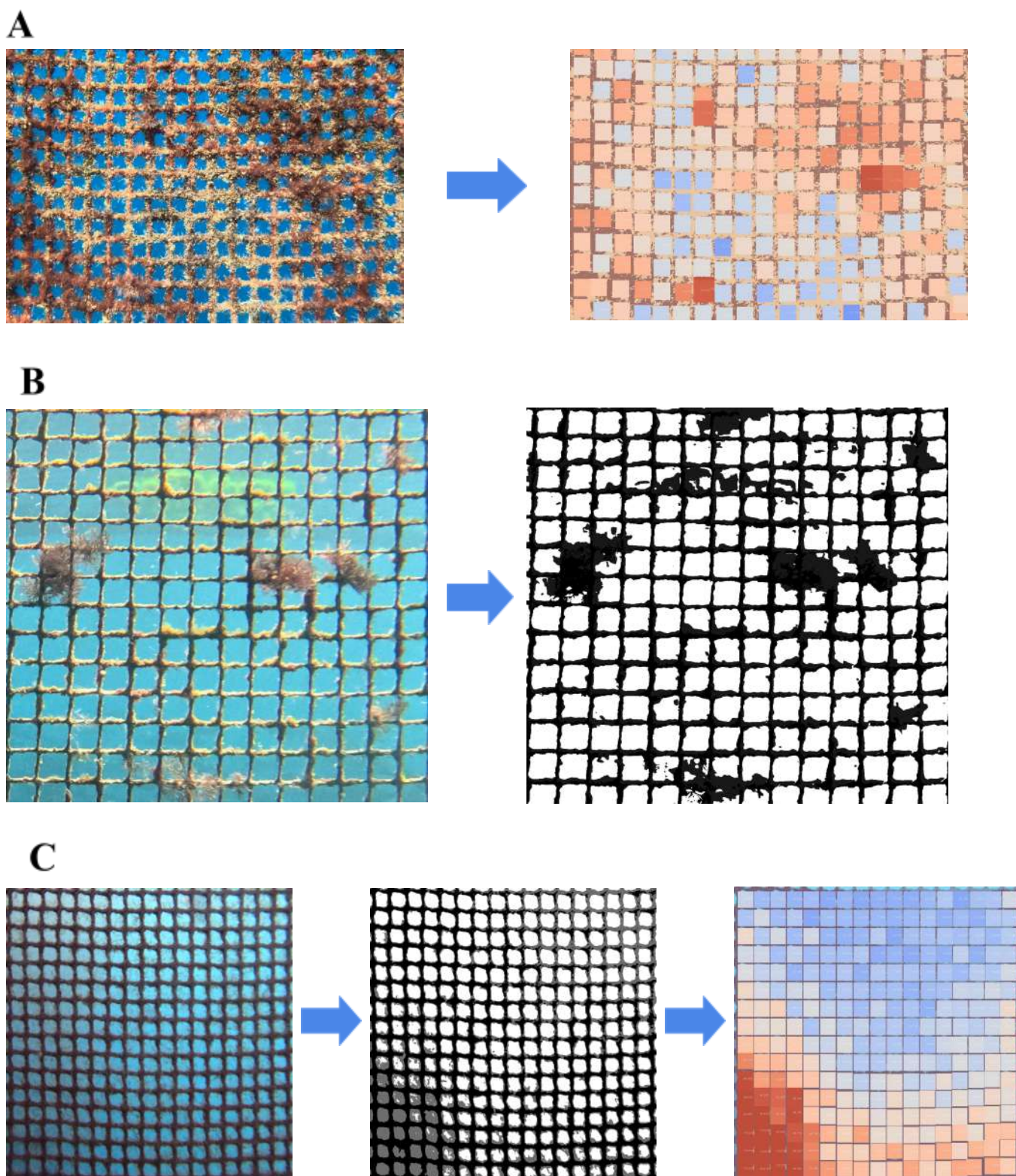


Figure 27. Trois résultats de traitement obtenus par l'algorithme. **A: Limites du postulat de base sur les mailles propres:** Aucune maille n'est pas recouverte de salissure, ainsi les rectangles sont tous trop petits et n'incluent pas une bonne partie du biofouling. D'autre part en A, le filet est légèrement courbé mais l'algorithme parvient à disposer les rectangles englobants aux bons endroits. **B: Indifférence aux couleurs des salissures:** Le filet est recouvert d'algues de couleurs différentes allant du bruns/rouge en passant par le jaune jusqu'au vert, cependant l'image binarisée montre bien que toutes ces algues ont bels et bien été prises en compte. **C: Problèmes avec la variation de couleur de l'eau:** Ici, on observe qu'une partie de l'image est plus ombragée que l'autre, ainsi, l'algorithme détecte à tort du biofouling

Robustesse de la Détection des Ouvertures

Malgré les défis mentionnés, l'algorithme parvient toujours à identifier correctement les ouvertures, sans confusion avec les zones d'encrassement. Indépendamment de la couleur de l'eau, l'ouverture reste toujours l'élément le plus "bleu" de l'image, ce qui permet à l'algorithme de la détecter avec constance. Cette robustesse dans la détection est essentielle pour garantir une analyse fiable des filets, même dans des conditions d'éclairage et d'environnement variées.

En conclusion, bien que l'algorithme présente une bonne adaptabilité à divers défis, certains aspects, tels que la détection des poissons proches, les filets très encrassés, et les variations de couleur de l'eau, méritent une attention particulière pour affiner encore plus l'algorithme.

IV- PISTES D'AMÉLIORATION:

Bien que les résultats obtenus apportent des éclaircissements sur les points forts et les limites de l'algorithme, plusieurs pistes d'amélioration demeurent à explorer pour en renforcer la robustesse. Il serait particulièrement pertinent de mener des tests supplémentaires sur un large échantillon de photographies de filets, dont l'encrassement aurait été préalablement noté par les plongeurs. Cette démarche permettrait de vérifier la cohérence entre la notation manuelle des plongeurs et les pourcentages d'encrassement générés par l'algorithme. De plus, à l'issue de cette phase de validation, il pourrait être envisageable de développer un module permettant à l'algorithme d'attribuer une note aux filets selon le même système utilisé par les plongeurs, en se basant sur les pourcentages calculés.

Les filets photographiés au cours de mon parcours ont déjà permis une comparaison préliminaire des pourcentages de recouvrement obtenus avec les notations manuelles des plongeurs. Les résultats montrent une certaine cohérence entre les deux évaluations, ce qui est encourageant si les travaux venaient à se poursuivre.

CONCLUSIONS

Mon stage chez Acquadea m'a offert une expérience riche, tant sur le plan technique qu'humain. En mer, j'ai eu l'opportunité de travailler aux côtés de l'équipe responsable des opérations sur le site, ce qui m'a permis d'acquérir des compétences pratiques essentielles, telles que le changement des filets, le déplacement des cages, le transfert des poissons et la prise de poids moyen. Ces tâches m'ont non seulement permis de mieux comprendre le fonctionnement quotidien d'une ferme aquacole offshore, mais elles m'ont également donné une appréciation plus profonde des défis logistiques et techniques liés à la gestion d'une telle installation.

Au bureau, j'ai concentré mes efforts sur le développement de l'algorithme de détection de l'encrassement des filets. Ce travail m'a permis d'approfondir mes compétences en programmation, notamment en Python, et de comprendre les subtilités du traitement d'images appliqué à un contexte aquacole.

D'un point de vue personnel, ce stage a été une expérience formatrice qui m'a permis de combiner mes connaissances académiques avec une application pratique dans un environnement professionnel exigeant. J'ai pu développer ma capacité à travailler en équipe, à m'adapter à des situations nouvelles et à gérer des projets de manière autonome. En somme, ce stage a renforcé mon envie de poursuivre une carrière dans le domaine de l'aquaculture.

Sources:

- Cardia, F. (2015). *Aquaculture Operations in Floating HDPE Cages : A Field Handbook*.
- Conides, A., Kallias, I., Cotou, E., Georgiou, P., Gialamas, I., & Klaoudatos, D. (2023). Preliminary Results on the Antifouling Potential of Copper Wire and Dyneema® Fiber Combined Twines for Aquaculture Net Cages. *WSEAS TRANSACTIONS ON ENVIRONMENT AND DEVELOPMENT*, 19, 607-612.
<https://doi.org/10.37394/232015.2023.19.59>
- Edwards, C. D., Pawluk, K. A., & Cross, S. F. (2014). The effectiveness of several commercial antifouling treatments at reducing biofouling on finfish aquaculture cages in British Columbia. *Aquaculture Research*, 46(9), 2225-2235.
<https://doi.org/10.1111/are.12380>
- Gansel, L. C., Plew, D. R., Endresen, P. C., Olsen, A. I., Misimi, E., Guenther, J., & Jensen, Ø. (2015). Drag of Clean and Fouled Net Panels – Measurements and Parameterization of Fouling. *PLoS ONE*, 10(7), e0131051.
<https://doi.org/10.1371/journal.pone.0131051>
- Greene, J. K., & Grizzle, R. E. (2007). Successional development of fouling communities on open ocean aquaculture fish cages in the western Gulf of Maine, USA. *Aquaculture*, 262(2-4), 289-301. <https://doi.org/10.1016/j.aquaculture.2006.11.003>
- Kristiansen, T., & Faltinsen, O. M. (2012). Modelling of current loads on aquaculture net cages. *Journal Of Fluids And Structures*, 34, 218-235.
<https://doi.org/10.1016/j.jfluidstructs.2012.04.001>
- Nobakht-Kolur, F., Zeinoddini, M., Harandi, M. A., Abi, F., & Jadidi, P. (2021). Effects of soft marine fouling on wave-induced forces in floating aquaculture cages : Physical model testing under regular waves. *Ocean Engineering*, 238, 109759.
<https://doi.org/10.1016/j.oceaneng.2021.109759>
- Qiu, W., Pakrashi, V., & Ghosh, B. (2020). Fishing net Health State estimation using underwater imaging. *Journal Of Marine Science And Engineering*, 8(9), 707.
<https://doi.org/10.3390/jmse8090707>
- Dürr, S., & Thomason, J. C. (2009). *Biofouling*. John Wiley & Sons.
- Agep, É. P. R. (s. d.). *Gloria Maris Groupe - accueil*. <https://www.gloriamarisgroupe.com/>

